FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 – SPECIAL TOPIC IN DATA ENGINEERING

SECTION 1

Tutorial 4: Claude & Pipeline

LECTURER: DR. ARYATI BINTI BAKRI

Data source:

https://www.youtube.com/watch?v=qGu7hDaJG1A **(ETL data pipeline with an AI Agent)**

GROUP 3

STUDENT NAME	MATRIC NO
ELIJAH SHE YU SHENG	A23CS0073
CHERYL CHEONG KAH VOON	A23CS0060
CHUA JIA LIN	A23CS0069

TABLE OF CONTENTS

1.0 What is AI-assisted data engineering

AI-assisted data engineering means using an AI Agent to help build a data pipeline through prompts instead of manually coding every step. In this tutorial, Nexla Express AI Agent was used to create a pipeline from WeatherAPI.com to a Snowflake table named WEATHER_DATA.The AI Agent helped create the WeatherAPI source, configure the API key, generate the Nexset, connect to Snowflake, and create the Snowflake sink. It also helped perform a transformation by converting FEELSLIKE_C from Celsius into Fahrenheit and storing the result in a new column named FEELSLIKE_F.

Therefore, this tutorial shows that AI-assisted data engineering can support the full ETL process, including extracting data from an API, transforming weather data, and loading the last result into Snowflake.

2.0 What is Claude/Ai agent use in the tutorial

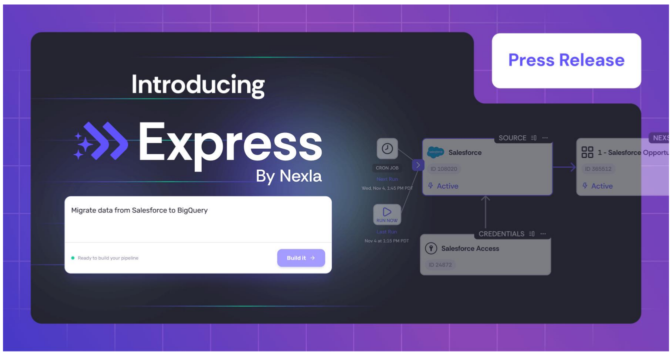

Figure 2.1: Nexla Express (Nexla, 2025)

In this tutorial, the AI agent act as an intelligence assistant and automatically performs multiple data engineering tasks through natural language. User only need to describe the requirement of pipeline instead of manually configuring connector, architecture and workflow. AI Agent was used inside Nexla Express to build the weather data pipeline through prompt messages. The AI Agent helped create the WeatherAPI.com source, collect the API key, set the weather location, generate the Nexset, configure the Snowflake credential, and create the Snowflake sink.

The AI Agent was also used to perform data transformation. It transformed the `FEELSLIKE_C` value from Celsius into Fahrenheit and stored the result in a new Snowflake column named `FEELSLIKE_F`. Therefore, the AI Agent helped complete the main ETL process, which includes extracting weather data from WeatherAPI.com, transforming the data, and loading it into the Snowflake `WEATHER_DATA` table.

3.0 Steps to do prompt and coding

This tutorial created a pipeline that fetch current weather data from WeatherAPI.com for Malaysia every 2 minutes, transforms the data to add a calculated Fahrenheit temperature field, and loads it into a Snowflake data warehouse table.

3.1 Prerequisite to Create Pipeline

Before creating the pipeline, a few configurations and settings need to be done.

1. **Obtain WeatherAPI.com API key**

   A WeatherAPI account was created to obtain an API key. The API key was used to fetch Malaysia (Kuala Lumpur) weather data from the website. Figure 3.1.1 shows the API key used in this tutorial.

Figure 3.1.1 API key for WeatherAPI.com

2. **Creating Snowflake WEATHER_DATA Table**

   A Snowflake table was created in this step. The table contains records of weather data from Kuala Lumpur, Malaysia. Each record represents a single observation and includes details about the location, weather conditions, and other environmental factors.

   The following shows the Snowflake configurations to store weather data from WeatherAPI.com:

   - Warehouse Name: ***COMPUTE_WH***
   - Database Name: ***MY_DATABASE***
   - Database Schema: ***PUBLIC***
   - Table Name: ***WEATHER_DATA***

   Figure 3.1.2 shows the table definition of the WEATHER_DATA table.

```sql
create or replace TABLE MY_DATABASE.PUBLIC.WEATHER_DATA (
    LOCATION_NAME VARCHAR(16777216),
    LOCATION_COUNTRY VARCHAR(16777216),
    LATITUDE FLOAT,
    LONGITUDE FLOAT,
    LOCAL_TIME VARCHAR(16777216),
    LAST_UPDATED VARCHAR(16777216),
    TEMP_C FLOAT,
    TEMP_F FLOAT,
    CONDITION_TEXT VARCHAR(16777216),
    CONDITION_CODE NUMBER(38,0),
    WIND_KPH FLOAT,
    WIND_DIR VARCHAR(16777216),
    PRESSURE_MB FLOAT,
    PRECIP_MM FLOAT,
    HUMIDITY NUMBER(38,0),
    CLOUD NUMBER(38,0),
    FEELSLIKE_C FLOAT,
    VISIBILITY_KM FLOAT,
    UV_INDEX FLOAT,
    FEELSLIKE_F FLOAT
);
```

Figure 3.1.2 Table Definition of WEATHER_DATA Table

3.2 Steps to Create Pipeline

The pipeline was built completely using prompt messages without a single line of code. The following clearly explains the steps of creating the weather data pipeline.

At the beginning, a prompt was entered into the Express AI agent to ask the agent to create a pipeline from WeatherAPI.com to the Snowflake table. The prompt used was "***I want to create a pipeline from the weather api at WeatherAPI.com to my snowflake table WEATHER_DATA. It should run every 2 minutes.***"

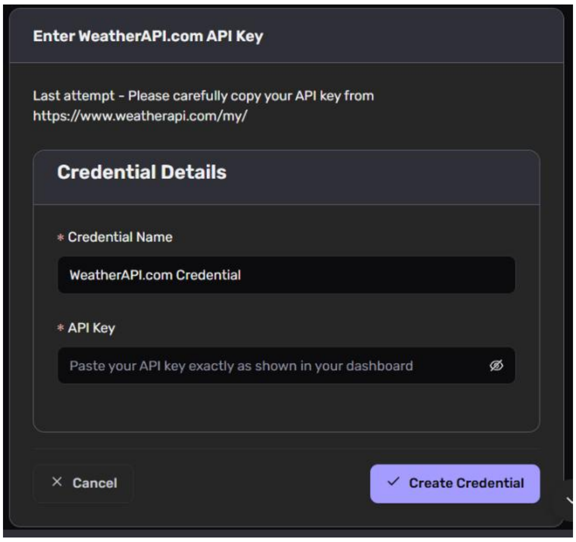

Figure 3.2.1 Form to Obtain API Key

The Express AI agent set up a **REST** (Representational State Transfer) connector for WeatherAPI.com to enable data fetching from the website. Figure 3.2.1 shows the AI agent was requesting API key input to create the **WeatherAPI.com Credential**.

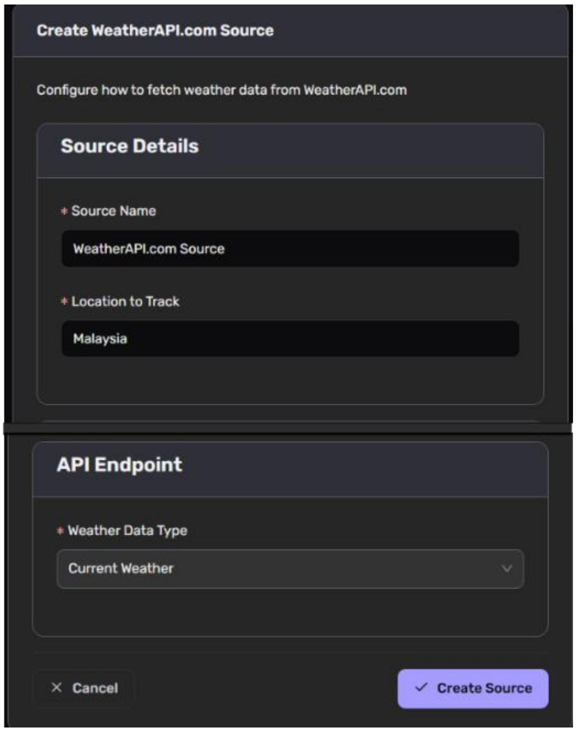

Figure 3.2.2 Form to Obtain Location to Track Information

Next, the **WeatherAPI.com Source** was created after the AI agent obtained the preferred location to track the weather data. Figure 3.2.2 shows the form prepared by AI agent to get the preferred location to track for the weather data pipeline.

After the WeatherAPI.com Credential and WeatherAPI.com Source were successfully created, a Nexset named **1-WeatherAPI.com Source** was created. Nextset is the logical representation of data. It was automatically generated by Nextla and acts as a data asset. It stores structured weather data after extraction from the source API and before loading into Snowflake table. It ensures that the data is cleaned, standardized, and ready for further processing or storage.

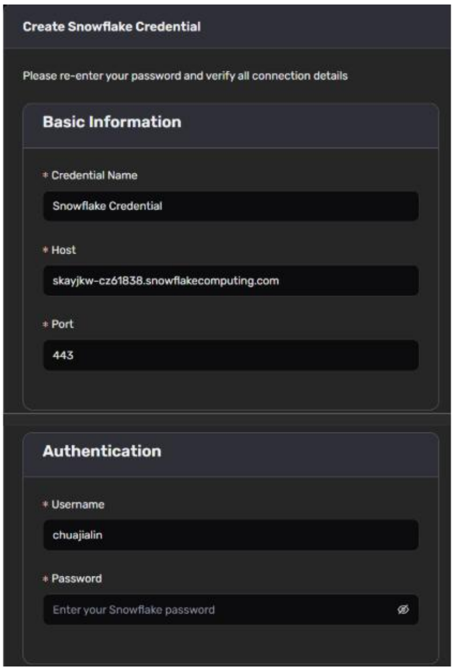

Figure 3.2.3 Form to Obtain Snowflake Credential for Basic Information and Authentication

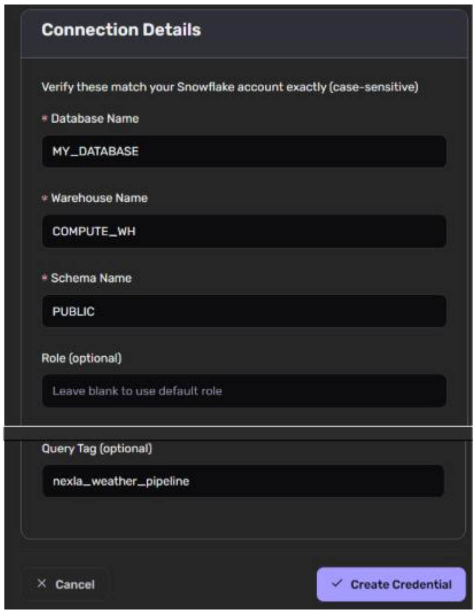

Figure 3.2.4 Form to Obtain Snowflake Credential for Connection Details

Then, the Express AI agent prepared a form to get the **Snowflake Credential**. Figure 3.2.3 shows the form to obtain Snowflake Credential for Basic Information and Authentication details, while Figure 3.2.4 shows the form to obtain Snowflake Credential for Connection Details. After submitting the form, the AI agent created the Snowflake sink and set up the sink configurations. Next, a **Snowflake WEATHER_DATA Sink** was created, and weather data were stored in the WEATHER_DATA table.

Currently, the table contains the FEELSLIKE_C variable. The transformation planned was to **extract FEELSLIKE_C** data and **convert** the data to **Fahrenheit** and **store** the converted data into a new column named **FEELSLIKE_F**. Therefore, a new column named FEELSLIKE_F with float data type was created in the WEATHER_DATA table.

Next, the prompt was entered into the Express AI agent to perform **data transformation** before loading data into the Snowflake WEATHER_DATA table. The prompt was ***"I created another column named FEELSLIKE_F in Snowflake WEATHER_DATA table. Create a transformation step that will take the field FEELSLIKE_C from the API, transform it into Fahrenheit and store it in the new column FEELSLIKE_F."***

The Express AI agent successfully **updated** the weather data pipeline. The pipeline successfully **fetched weather data** from the WeatherAPI.com website, **transformed** the **FEELSLIKE_C data** into **Fahrenheit**, and **stored** the **transformed data** into a new **FEELSLIKE_F** variable. Figure 3.2.5 demonstrates that the pipeline ran successfully because the FEELSLIKE_F column was not null anymore. The successful overall confirmation of the conversion logic in the FEELSLIKE_F column has been executed correctly. The pipeline continues to extract real-time weather data from WeatherAPI.com, performing conversion from Celsius to Fahrenheit and load the transformed data into snowflakes. This demonstrates that the ETL workflow runs successfully without manual intervention. Figure 3.2.6 illustrates the complete weather data pipeline architecture.

FEELSLIKE_C	VISIBILITY_KM	UV_INDEX	FEELSLIKE_F
29.4	5	0	null
29.4	5	0	null
29.4	5	0	null
41	10	0	105.8
41	10	0	105.8
41.3	10	0	106.34
41.3	10	0	106.34

Figure 3.2.5 Data Records in WEATHER_DATA Table

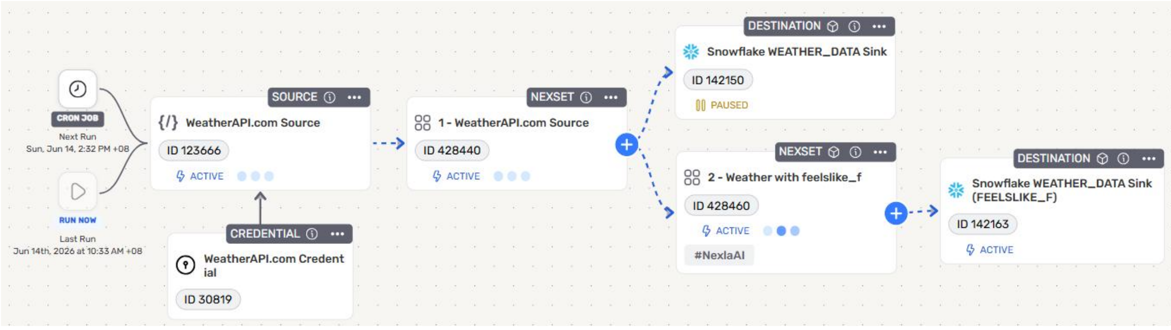

Figure 3.2.6 Pipeline Architecture

4.0 Use Case Diagram

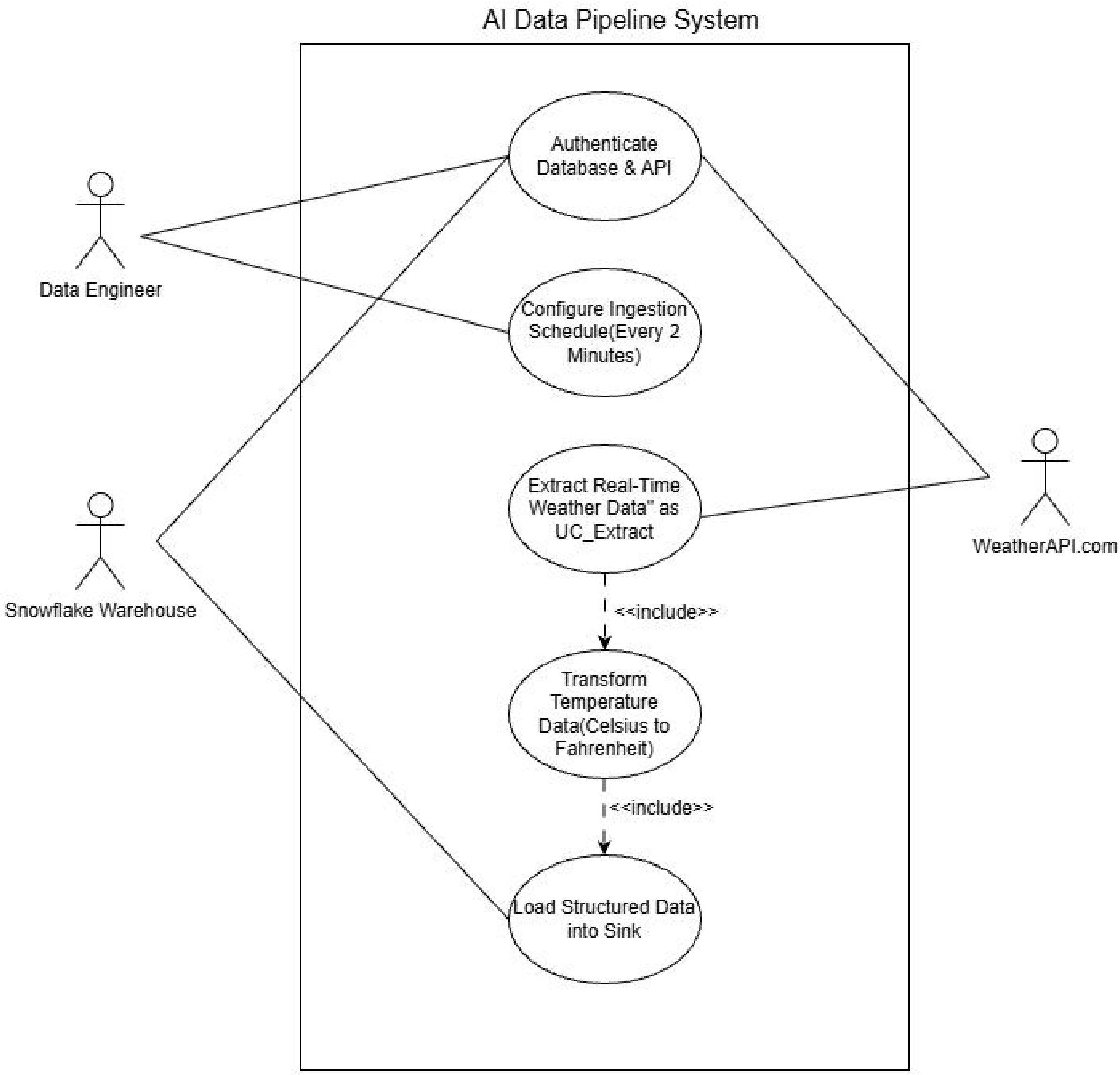

Figure 4.1 AI data pipeline System

Figure 4.1 shows the use case diagram for AI data pipeline system. The use case illustrates the interaction between the actor, and the artificial intelligence assisted weather data pipeline system. It shows how the data engineers, WeatherAPI.com and Snowflake Warehouse participate in the ETL workflow managed by Nexla Express AI agent.

The first actor is **Data Engineer,** who is responsible for configure pipelines, provide the required credentials and manage the overall workflow. Second is **WeatherAPI.com**, which are the external data sources that request real-time weather information through API. Next is Snowflake Warehouse to store the destination database that has processed weather data for future analysis.

The Configure Ingestion Schedule use case allows data engineers to define the operating frequency of the data pipeline. In this tutorial, the workflow is configured to run automatically every two minutes to ensure the continuous collection and processing of the latest weather information.

The Authentication Credentials use case is responsible for verifying the WeatherAPI key and Snowflake connection details before any data processing activity begins. This ensures that secure communication can be established between the source system and the target system.

Extract Real-time Weather Data use cases to retrieve current weather information from WeatherAPI.com through API requests. The extracted data is used as the original input for the subsequent conversion and loading process.

The Transform Temperature Data use case converts the FEELLIKE_C value to FELSLIKE_F as part of the data conversion process. This use case has a <include> relationship with the extraction process, because whenever the weather data is retrieved, the conversion step must be performed.

The Load Structured Data into Sink use case save the converted weather data in the WEATHER_DATA table in Snowflake. This use case is included in the conversion process, because the converted data must be loaded into the target database for storage and future analysis. <include> Relationships represent mandatory activities in the ETL workflow. They ensure that the extracted data is converted before loading into the snowflake warehouse to maintain a consistent and structured data processing sequence.

5.0 Reflection

ELIJAH SHE

This tutorial helped demonstrate how an AI Agent can support the process of building a data pipeline from WeatherAPI.com to Snowflake. Through this project, learned how to use prompt messages to create a data source, configure credentials, connect to Snowflake, and perform a simple transformation by converting FEELSLIKE_C into FEELSLIKE_F. The tutorial also showed that AI can speed up data engineering work, but the final output still needs to be checked carefully to make sure the data is loaded correctly. Overall, this task improved the understanding of API connection, pipeline creation, data transformation, and Snowflake data verification.

CHERYL CHEONG KAH VOON

This tutorial provides a better understanding of how artificial intelligence can support data engineering by automating data pipelines and reducing manual coding. The workflow generated by AIU agent successfully connects WeatherAPI data to Snowflake and simplifies the data ingestion process.

Several challenges were encountered during the implementation process including the snowflake connection problem and the failure to load the conversion data into the target table. These problems highlight the limitations of AI-generated workflows, especially when troubleshooting and modifying pipeline components.

In general, the tutorial shows the advantages and limitations of artificial intelligence-assisted data engineering. Future implementation can be improved through more detailed tips and additional data verification processes to ensure accurate data loading and conversion.

CHUA JIA LIN

The process of creating a pipeline using the Express AI agent provided valuable learning in AI-assisted workflows and data engineering concepts. Through this tutorial, a deeper understanding of data pipeline processes was developed, and new hands-on experience in building a pipeline using prompts only, without writing a single line of coding was gained. This experience is very meaningful because the use of AI agents has become the latest trends in industries, showing how automation and AI-driven solutions are transforming modern development practices.

The use of the Express AI agent offered significant benefits in supporting technical development. Most tasks can be completed by prompting the AI agent, which significantly improved convenience, productivity, and efficiency. However, one disadvantages of using an AI agent is that the generated output can be difficult to modify when issues were found in the workflow. For future improvement, better prompting skills should be applied to ensure that the requirements are clearly understood by the AI agent so that more reliable and accurate outputs can be produced.